Computer Science & IT



C Programming

Data Types and Operators

Lecture No. 05



By- Abhishek Sir

Recap of Previous Lecture



- Topic — Expression ✓
- Topic — Precedence ✓
- Topic — Associativity ✓
- Topic
- Topic

Slide

Topics to be Covered



Topic Relational Operator

Topic Logical Operator

Topic

Topic

Topic

Slide

$12 \leftarrow$
8

$2 \% 8$
$-2 \% 8$
$2 \% -8$
$-2 \% -8$

Q: 0 Re: 2
Q: 0 R -2
Q: 0 R : : 2
Q: 0 R : -2

```
int x;
 x= -2 + 11 - 103 * 2 % 3 / 2;
printf("%d",x);
return 0 ;
}
```

(a) 6
(b) 7
(c) 8
(d) 9

$= -2+11-\underline{206\%3/2}$

$= -2+11-2/2$

$= -2+11-1$

$= 9-1$

$= 8$

$$
\begin{array}{r|l}
3 & 206 \\
 & 18 \\
\hline
 & 26 \\
 & 24 \\
\hline
 & 2 \leftarrow
\end{array}
\qquad 68
$$

Slide

```
int main(){
int x;
x= 2 * 3/4+4/4+5/8- 8*4;
printf("%d",x);
return 0 ;
}
```

____

(a) 60
(b) 30
(c) -30
(d) 90

6/4+1+0-32

[c]

1+1+0-32

2+0-32

2-32 = -30 ✓

Slide

```c
#include<stdio.h>
   int main(){
   int x;
   x=  -3%2*4%3/8+3;
   printf("%d",x);
   return 0 ;
   }
```

The value of x is

(a) 3
(b) -3
(c) 4
(d) 5

$-3\%2*4\%3/8+3$

$-1*4\%3/8+3$

$-4\%3/8+3$

$-1/8+3 = 0+3 = 3$

Slide



Toipc: Scope of a variable

```
int main() {

}
```

Scope :- variable - declaration
assign, use

Slide



Toipc: Scope of a variable

The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable and it is treated as an undeclared identifier.

Undeclared.



Toipc:Scope of variable

```
#include<stdio.h>
int  main {
     {
          int a =10 ;
          printf("%d"; a); // will print the 10
     }


}
```

block

Slide



Toipc:Scope of variable

```
#include<stdio.h>
int  main() {
   {
     int a =10 ;
   }
   printf("%d", a); // will not print 10
   return 0 ;
}
```

Error out of Scope

Slide



Toipc:Scope of variable

```
#include<stdio.h>
int  main() {
   {
          int a =10 ;
          printf("%d", a); // will print 10
   }

   return 0 ;
}
```

Lexical Scope :

if declaration of a variable not found within a block the declaration of variable Searched in outer block.

This process is repeated until definition ~~det~~ declaration of variable found. if Not Found the error.



Toipc: Question

```
#include<stdio.h>
int x = 40;
int  main() {
       int x = 30;
       {
           int x = 20;
           {
                int x = 10;
                printf("%d", x);
           }
       }
      return 0 ;
}
```

global

visible to all

Slide



Toipc: Relational Operator

$>$, $<$, $>=$ $<=$, $==$ $!=$

(1) 10>5 . 1 ← if statement true

0 ← if statement false

1

(2) 10<5 Ans 0

Slide

(3) 10 >= 7, 10 >= 10

T. (1) True - 1

(4) 4 <= 5. print 1

(5) 4 == 5 (4 is exactly equal to 5)

Answer is false → print 0

(6) != 4!=5 true, point 1

* 5!=5 – false, point 0



Toipc: Relational Operator

>	<	<=	>=	==	!=
Less than	Greater than	Greater than equal to	Less than equal to	Exactly equal	Not equal to

output 1 true
0 - false

Slide



Toipc: Relational Operator Example

```
#include<stdio.h>
int x = 40;
int  main() {
      printf("%d\n", 30>40);
      printf("%d\n", 30>=40);
      printf("%d\n", 30==40);
      printf("%d\n", 30!=40);
      printf("%d\n", 40!=30);
      printf("%d\n", 40==40);
      printf("%d\n", 50>50);
      printf("%d\n", 50<=50);
      return 0 ;
}
```

0
0
0
1
1
1
0
1

00011101

10>=7

greater or equal

Slide

\n print the next statement in new Line.



Toipc: Relational Operator Precedence

> <   >= <=
==, !=

Left to Right

10 == 4 < 5

printf("%d", 10 == 4 < 5);

10 == 1 ✓

Ans 0

Slide

printf("%d", (4<=5)<0);

Left to Right

Associative.

(4<5)<0

1<0

0

(4<5)<0

1<0

0

printf("%d", 4>=3 == 2<=3);

1 == 1

1



Toipc: Relational Operator Precedence

– unary –

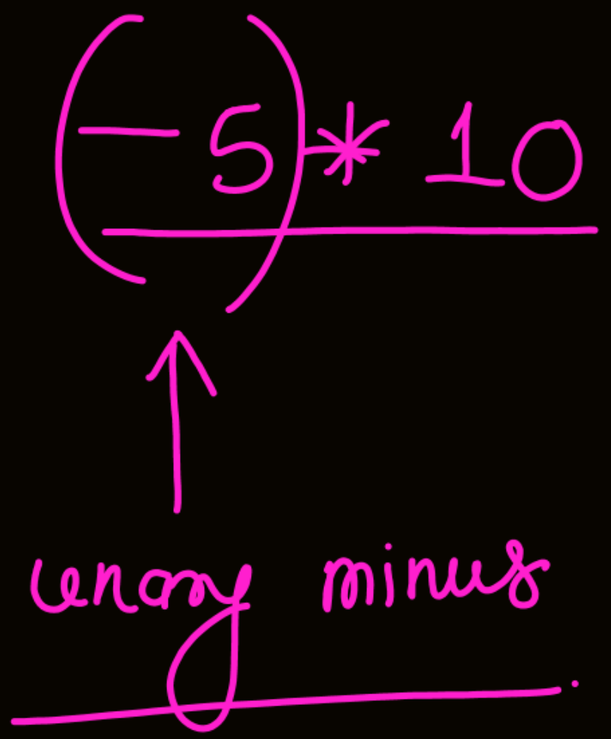
Aorthmetic

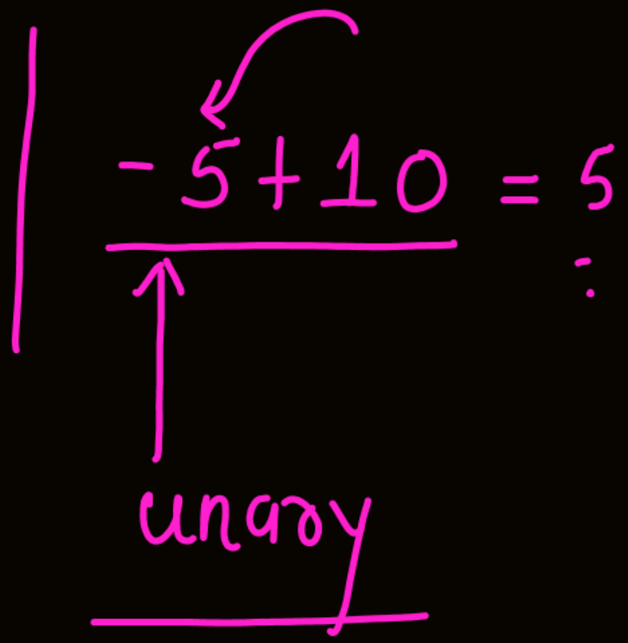
3	* / %	Multiplication, division, and modulus	left to right
4	+ -	Addition and subtraction	left to right
6	< <= >>=	Relational less than and less than or equal to Relational greater than and greater than or equal to	left to right
7	== !=	Relational equal to and not equal to	left to right

23+24 <= -5+20

47 <= 15 answer false.

Slide


(-5)*10
unary minus


-5+10 = 5
unary



Toipc: Logical Operator &&

if( )

1. AND
2. OR
3. NOT

printf("%d", 4>5 && 5<3)

0 && 0

0

Truth table

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Slide



Toipc: Logical Operator (AND) &&

```
#include <stdio.h>
int main(void) {
    int a = 20;
    int b = 30;
    printf("%d", a > 10 && b > 10);
}
```

20>10 30>10

1 && 1 = 1

Slide



Toipc: Logical Operator (OR) ||

```c
#include <stdio.h>
int main(void) {
  int a = 20;
  int b = 5;
  printf("%d", a > 10 || b > 10);

}
```

20>10

5>10

1 || 0 ⇒ 1

Logical OR ||

A	B	A || B
0	0	0
0	1	1
1	0	1
1	1	1

Slide



Toipc: Logical Operator (NOT) !

A	!A
0	1
1	0

Slide



Toipc: Logical Operator (NOT) !

```
#include <stdio.h>
int main(void) {
  int a = 20;
  int b = 5;
  printf("%d", !(a < 10 ));
}
```

!(20<10) = !(0) = 1

&&
||
Not
} logical

Slide

1. Arithmetic — * / % + -

2. Relation — > < >= <= == !=

3 Logical — && || !

Negation

Negation - unary

minus - unary



Toipc: Precedence

3	* / %	Multiplication, division, and modulus	left to right
4	+ -	Addition and subtraction	left to right
6	< <= > >=	Relational less than and less than or equal to Relational greater than and greater than or equal to	left to right
7	== !=	Relational equal to and not equal to	left to right
11	&&	Logical AND	left to right
12	||	Logical OR	left to right

Arthmetic

Relation

Logical

Slide



Toipc: Question

```
int main()
    {
        int x = 1, y = 0, z = 0;
        int a = x && y || z+1;
        printf("%d", a);
    }
```

A. 6
B. 5
C. 0
D. 1

a = x && y || z+1

1 && 0 || 1

0 || 1 = 1

Slide



Toipc: Question

```
#include <stdio.h>
    void main ()  {
        int x = 1, y = 0, z = 0;
        int a = x && y && z+1;
        printf("%d", z);
    }
```

A. 6
B. 5
C. 0
D. 1

Slide



Toipc: Precedence

3	* / %	Multiplication, division, and modulus	left to right
4	+ -	Addition and subtraction	left to right
6	< <= > >=	Relational less than and less than or equal to Relational greater than and greater than or equal to	left to right
7	== !=	Relational equal to and not equal to	left to right
11	&&	Logical AND	left to right
12	||	Logical OR	left to right



2 mins Summary

&pic

&pic

&pic

&pic

&pic

Slide

THANK - YOU

Slide